

mise_en_oeuvre

QA + Scrum like + responsable produit (segmente les doc) + voir pour du pentest

Contexte:

Concevoir et mettre en œuvre un système d'automatisation pour maisons connectées qui soit à la fois robuste, sécurisé et facile à utiliser.

Périmètre du projet

Client cible: Maison de 2 étages 100m² avec jardin de 40m²

Ce dont la maison peut disposer :

- Portail électrique
- Volets Electrique
- Jardin (arrosage)
- Chauffage
- Eclairage (intérieur et extérieur)

Budget total du client: 2000e - 5000e

Expression du besoin

Pouvoir manipuler à distance les lumières, la température des pièces, la sécurité, les appareils intelligents tel que la télévision ou une radio ainsi que l'ouverture des volets et d'un portail automatique

EB-01: Administration de l'application

- L'utilisateur souhaite pouvoir administrer l'utilisation de ses appareils connectés sur une applications mobile. Un agencement ergonomique de différents panneaux dans son espace doit lui être proposé afin de s'adapter à son matériel et son installation. Tout cela offrira une modularité importante pour l'utilisateur tout en contraignant sa façon de personnaliser l'application pour qu'elle reste ergonomique

EB-02: Gestions des droits et authentification

- L'utilisateur souhaite pouvoir partager l'accès au système domotique avec d'autres personnes (membres de la famille, invités), tout en gardant le contrôle total sur qui peut manipuler quels équipements, afin de garantir la sécurité du domicile.

Prise en charges des objects connectés

EB-03: Gestion des éclairages

- L'utilisateur souhaite pouvoir gérer les éclairages connecté à l'aide d'une application plutôt qu'avec un ensemble d'interrupteurs afin d'améliorer le confort d'utilisation (ergonomie) et d'optimiser la consommation électrique du domicile

EB-04: Gestion des systèmes de chauffage

- L'utilisateur souhaite pouvoir gérer les systèmes de chauffage à travers une application afin d'améliorer la gestion de sa consommation énergétique. Il aura également l'usage de chauffer certaines pièces sans y être physiquement

EB-05: Gestion des volets électriques

- L'utilisateur souhaite pouvoir gérer l'ouverture et la fermeture de ses volets électriques sans avoir à être physiquement présent dans son habitat. En cas de dysfonctionnement du volet, l'utilisateur souhaite en être informé sur son application.

EB-06: Gestion des systèmes d'arrosages automatisés

- L'utilisateur souhaite pouvoir gérer son système d'arrosage intelligent en planifiant des plages horaires d'arrosage. Il doit également pouvoir accéder à sa consommation d'eau dû à l'arrosage et pouvoir veiller à ne pas surconsommer

EB-07: Gestion du système de portique électrique

- L'utilisateur souhaite pouvoir ouvrir son portails à distance à travers son smartphone sans avoir à être physiquement présent dans son habitat

EB-08: Gestion des caméras de surveillance

- L'utilisateur souhaite pouvoir accéder à ses caméras de surveillance pour visualiser ce qu'il peut se passer devant les différents postes de vidéo surveillance lors de son absence. Il attend une sécurité particulière pour éviter les fuites de données ou la prise de contrôle par hacking de ses postes de video surveillance

Spécifications fonctionnels

SF-01 Authentification & Autorisation

- SF-1.1 : L'utilisateur doit pouvoir se connecter à son application de gestion de domotique de manière sécurisée.
- SF-1.2 : L'utilisateur doit pouvoir réinitialiser son mot de passe en cas d'oubli
- SF-1.3 : L'utilisateur doit pouvoir mettre en place une authentification renforcé à double facteur accessible à l'aide d'application tierces

SF-02 Configuration de l'application et prise en main

- SF-2.1 : L'utilisateur doit pouvoir configurer son espace en définissant différents plateaux qui rassembleront des fonctionnalités par panel. Un plateau peut contenir différents panels, chacun dédié à une fonctionnalité.

- SF-2.2 : Les plateaux sont limités au nombre de 6 pour ne pas surcharger l'application en informations

- SF-2.3 : Les plateaux sont limité à 4 panels de gestions d'objets connectés, chacun.

SF-03 Eclairages & personnalisation

- SF-3.1 : L'utilisateur doit pouvoir régler l'intensité des lumières si ses éclairages sont réceptifs aux variations de luminosité. Sinon cela ne doit pas avoir d'impact sur le bon fonctionnement des éclairages

SF-04 Systèmes de chauffage

- SF-4.1 : L'utilisateur doit pouvoir choisir une consigne en degrés Celsius (ex: entre 10°C et 30°C) pour chaque appareils

SF-05 Volets électriques

- SF-5.1: L'utilisateur doit pouvoir ouvrir et fermer ses volet à l'aide d'un pourcentage lui permettant de connaître le taux d'ouverture ou de fermeture de chaque équipement

SF-06 Arrosages automatisés

- SF-6.1: L'utilisateur doit pouvoir choisir l'intensité de l'arrosage automatique
- SF-6.2 : L'utilisateur doit pouvoir changer de mode d'arrosage en fonction des différents équipement possible a paramétrer

SF-07 Portique électrique

- SF-7.1 : L'utilisateur doit pouvoir choisir s'il ouvre le grand portique ou le petit portique. L'application doit s'adapter en fonction du matériel que possède l'utilisateur

SF-08 Caméras de surveillance

- SF-8.1: L'utilisateur doit pouvoir rejouer la vidéo surveillance des caméras pendant pendant au moins 7 jours
- SF-8.2 : Un bouton doit permettre de télécharger des plages horaires de vidéo surveillance

SF-09 : Gestion commune des objets connectés

- SF-9.1 : L'utilisateur doit pouvoir assigner jusqu'à 10 équipements connectés maximum par panel, quel que soit leur type.
- SF-9.2 : L'application doit afficher en temps réel l'état d'activité (En ligne / Hors ligne / Allumé / Éteint) de chaque équipement configuré.
- SF-9.3 : L'utilisateur doit pouvoir ordonner l'allumage ou l'extinction de n'importe quel équipement compatible, de manière individuelle ou groupée.
- SF-9.4 : Les appareils doivent être programmables sur des plages horaires qui peuvent se répéter en fonction de plusieurs semaines types

SF-10 Surveillance de l'état des objets connectés (monitoring)

- SF-10.1 : Un suivi du temps d'activation en continue doit être affiché sur le détails de chaque appareil
- SF-10.2 : Un journal des logs doit être accessible par l'administrateur de l'application (le client final).

- SF-10.3: Un backoffice permet à l'administrateur de créer de nouveaux comptes avec différents droit d'accès aux appareils.

Spécification techniques & ADR (architecture decision record)

Fonctionnement de la domotique

Un serveur central est connecté a tous les appareils. Il permet de se connecter avec des identifiants sécurisés afin d'envoyer des signaux aux autres appareils.

Un serveur MQTT permet de recueillir tous les signaux des différents appareil

Spécification matériels

ST-01 Unité centrale:

- Raspberry Pi 4, processeur ARM, 4 Go RAM [lien d'achat](#) 169€
- Alimentation Officielle 15.3W USB-C pour Raspberry Pi 4 / 400 [lien d'achat](#) 9.60€
- Raspberry Pi 4 Argon ONE V2 cooling case [lien d'achat](#) 24.90€
- Disque dur SSD Vi550 S3 - 1 To (pour la vidéo) Verbatim [lien d'achat](#) 49.99€
- Contrôleur radio domotique - SONOFF ZigBee 3.0 USB Dongle Plus, EFR32MG21 Coordinator, Universelle USB ZigBee Hub, Passerelle ZigBee pour Home Assistant [lien d'achat](#) 19.90€
- StarTech.com Câble d'extension USB 2.0* actif 1,80 m [lien d'achat](#) 6.23€
 - ADR: pour contrer **l'interférence de radiofréquence (RFI)** générée par la norme USB 3.0 et pour contrer le conflit avec la fréquence qu'utilise Zigbee, ***Le chevauchement des fréquences**
- Câble Ethernet RJ45 [lien d'achat](#) 3.72€
- Onduleur UPS Anti court circuit [lien d'achat](#) 27.99€

Coût d'achat du serveur central sans l'installation: 283,34 €

[Compatibilité avec ZigBeeToMQTT](#)

Spécifications software

ST-02 Architecture du serveur central

- ST-2.1.0 **Choix du backend et du langage** : Golang
 - ADR : Accessible en terme de difficulté. Très léger. Go produit un binaire statique unique. Le conteneur final ne contenant ni shell ni dépendances système ce qui empêche l'exécution de code arbitraire ou l'injection de commandes système en cas de compromission locale ou distante.
 - ST-2.1.2 **Choix du framework pour l'API** : API REST en net/http avec GoRM
 - ST-2.1.3 **Authentification et Autorisation** : RBAC, refresh token. 2FA (en option pour l'utilisateur)

- ST-2.1.4 **Streaming vidéo des caméras de surveillance** : go2rtc, utilisation de webhook pour intercepter les appels webRTC
 - ADR l'architecture webRTC devra être en SFU (Selective Forwarding Unit). C'est la plus optimisé, permet de ne rien calculer sur le serveur et d'avoir beaucoup de destinataire si besoin
- ST-2.2 **Base de données locale**: Base de données postgres pour surveiller l'accès et l'état de tous les composants liés à l'application de domotique.
- ST-2.3.0 **Frontend** : Application en React Native
 - ADR: L'application devra être installée chez le client en APK et disponible sur l'app store 1 ans après la livraison de l'application
 - ST-2.3.1 **Les vues** : L'application doit comporter:
 - Un écran d'authentification
 - Un écran de backOffice pour le super admin de l'application
 - Un écran de gestions de plateau
 - Un écran de gestion de panel, pour chaque plateau
 - Un écran de gestion de panel pour chaque panel
 - Un écran pour contacter le support technique en cas de panne
- ST-2.4 **Serveur de messagerie MQTT** : Le rôle du serveur sera de recueillir l'ensemble des messages publié par l'IOT qui seront traités par le serveur backend. Utilisation de la distribution Mosquitto
 - ADR : Serveur conventionnel utilisé dans le cadre de l'IOT pour rester résilient face au traitement des données.
- ST-2.5 **Serveur de traduction réseau** : Zigbee2MQTT
 - ADR: pour traduire le flux zigBee entrant en JSON pour le serveur MQTT
- ST-2.6 **Virtualisation des technologies** : Utilisation de docker-compose pour monter tous les conteneurs de l'application.
 - ADR: Afin d'obtenir un software facile a déployer et compatible avec les environnements Linux
 - MQTT Mosquitto
 - Postgres
 - Dockerfile pour les binaires de Go
 - Zigbee2MQTT
 - ADR: traduction IOT vers JSON pour serveur MQTT
- ST-2.7 **Exposition serveur** : Exposition du port 3333 à travers la box internet
 - ADR: Permet d'accéder au serveur depuis l'extérieur de l'habitat
 - ST-2.7.1 Traefik : Configuration d'un serveur Traefik pour Docker
 - ADR : Permet d'éviter d'accéder à chaque serveur docker individuellement. l'API Go et le flux go2rtc seront exposés sur le réseau internet en ouvrant le port 443

sur la box internet.

Estimations des coûts

Trois coûts sont à distinguer pour le client final.

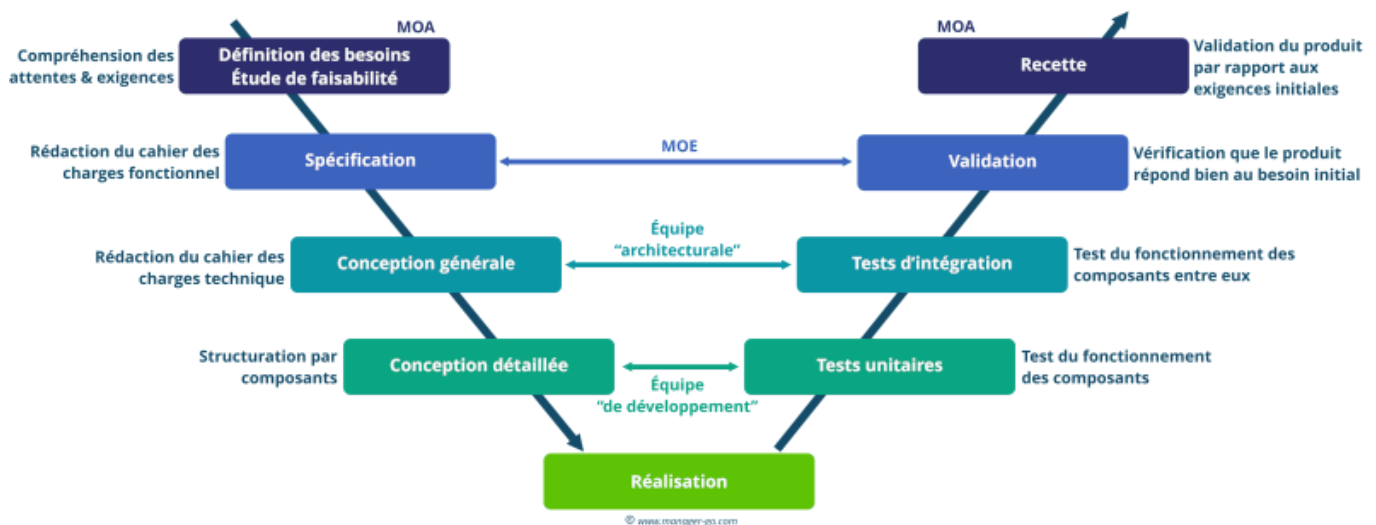
- L'achat du matériel de domotique (il peut déjà être présent pour certains clients)
- Le coût de l'installation par l'entreprise de domotique
- Le logiciel permettant d'accéder aux différents appareils (coût initial ou par abonnement)

Conception générale

Organisation et Pilotage du Projet

Afin d'assurer une forte qualité logiciel dans le domaine critique des objets connecté dans la domotique, nous allons travailler en extreme programming. Cela implique différentes étapes dans la conception et la réalisation des tâches à venir.

Le rythme de rencontre avec le client est organisé sur un cycle de 2 semaines



Répartition des rôles:

- Développeur back-end
- Développeur Front-end
- Designer UX/UI
- Testeur QA

Workflow en Extreme Programming.

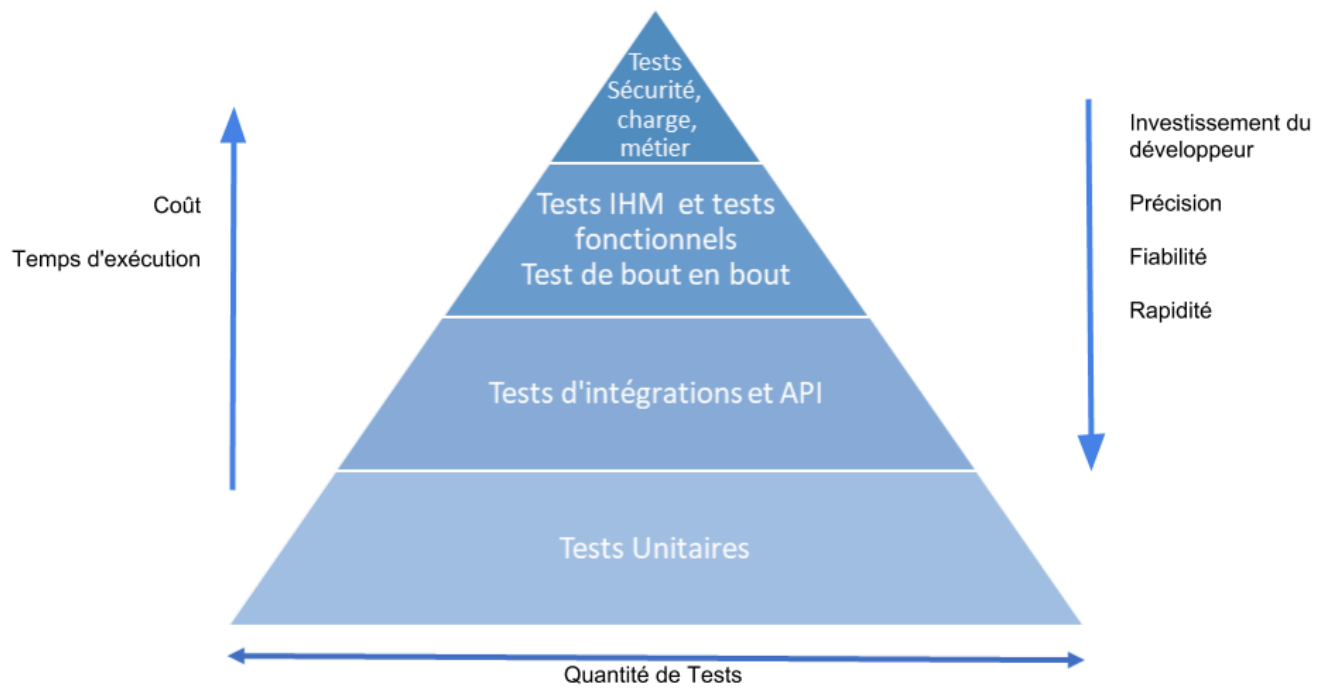
Etant donné que nous intervenons sur des protocoles réseaux particuliers nous n'allons pas travailler en Agile Scrum mais plutôt en Agile Extreme Programming afin de s'assurer que nous

avançons sans aucun bug et avec une très bonne compréhension globale du projet et des composants développés.

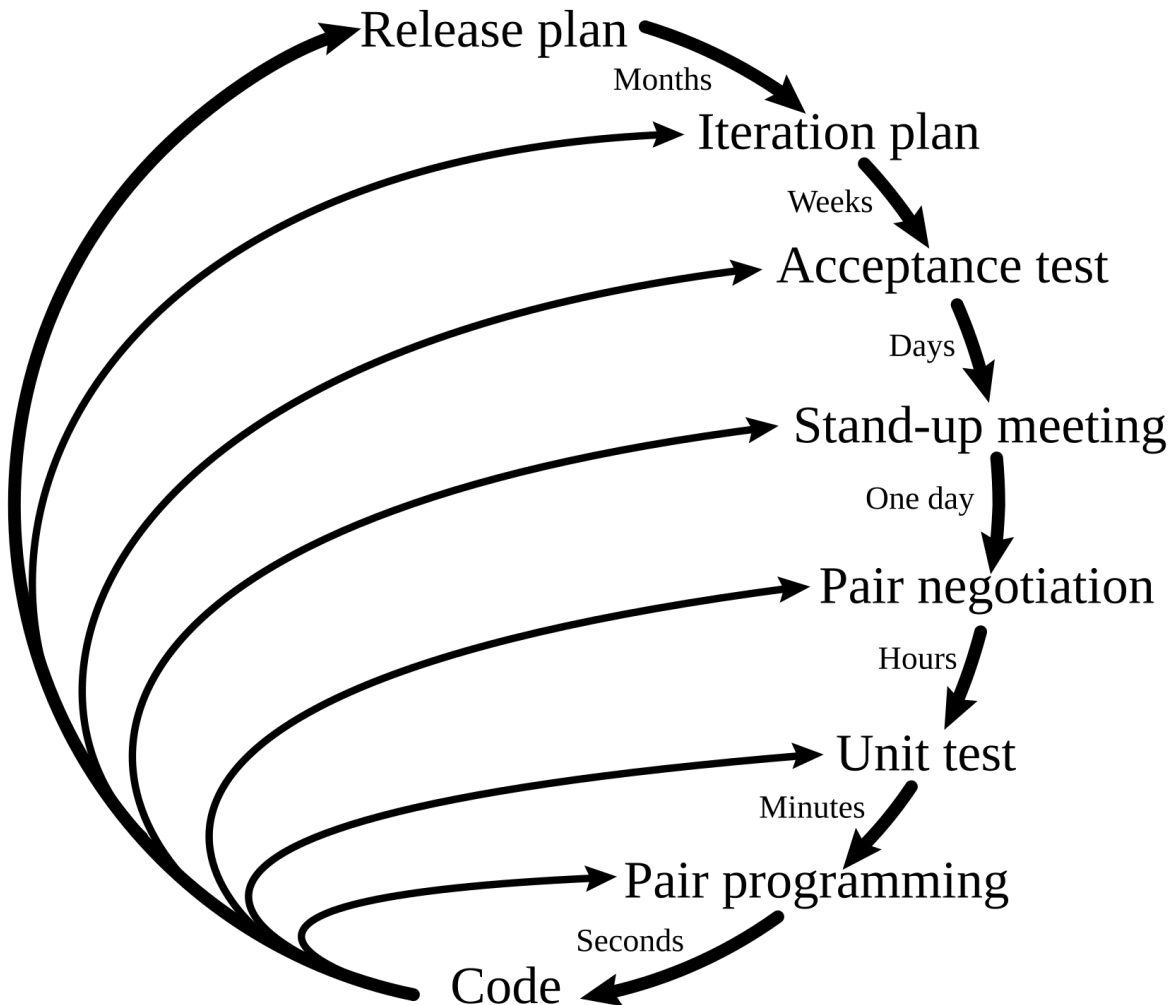
- Le Client (entreprise de domotique) en réunion avec le reste de l'équipe de développement devra valider la conception des User stories qui seront la base principale pour exécuter les tâches de développement. Le testeur QA en collaboration avec le développeur backend et parfois le développeur frontend sur les fonctionnalités correspondantes, auront la charge de rédiger des scénarios en Gherkins en Behavior-Driven Development pour chaque user story, et en complément d'une définition of done.
- En terme de conception il faudra s'en tenir strictement à la portée des tâches qui vous seront attribuées et ne jamais anticiper le travail futur sur vos tâches actuelles. C'est comme cela que nous pourrions nous assurer de la validité de chaque tâche sans créer du désordre.
- Les développeurs devront travailler en binôme (Pair programming) afin de s'assurer de la validité de la conception et de la réalisation du code. Les bugs devront se résoudre en équipe. Tout le développement devra être piloté par les Tests (Test-Driven Development - TDD) et respectent des normes de codage strictes. Les scénarios Gherkins vous guideront sur le TDD pour connaître les cas de tests à développer
- A chaque avancée nous exécuteront les tests unitaires ainsi que les tests d'acceptations pour valider la non régression du code
- Si certains tests sont invalidés, les développeurs devront les retravailler. Cela sera établi après une réunion rassemblant l'équipe afin d'établir un planning et revoir les priorités des tâches à venir
- Après la validation des tests unitaires et d'acceptations, le code doit être livré en continue et fusionné. Cela permettra de remonter plus haut dans les tests de validations ainsi que d'observer l'assemblage de l'application avec le client.
- Chaque nouvelle réunion avec le client mènera à un nouveau cycle de travail reprenant tous les points précédents

Stratégie de tests

La stratégie de test est présente pour répondre aux attentes de la qualité logiciel induite par le modèle du cycle en V. Chaque étape de conception et de réalisation devra correspondre à un type de test permettant de valider la qualité logiciel attendu.



Planning/feedback loops



Chaque étape du projet devra correspondre à un certains niveau de tests. Aussi, l'assemblage d'un cumul de petits composant devra donner lieu a des tests fonctionnels, puis end2end et enfin de charge et de sécurité. Ce qui rend le Peer Programming d'autant plus important car il faudra s'assurer de ne jamais rien oublier à la fois dans le déploiement des fonctionnalités et leur dépendances. C'est ainsi que nous assurerons la qualité logiciel en testant tous les cas possibles et en ne cassant jamais la chaîne d'assemblage des composants.

Chaque cycle de test devra donner lieu a un cahier des recettes.

Template du cahier des recettes :

1. Sélectionner une exigence

- Rappeler les spécificités fonctionnelles auxquels elles se rapportent.
- Rappeler les spécificités techniques qui seront abordées au cours du développement.

- Expliquer pourquoi ce cahier de test est rédigé et à quel(s) niveau(x) de test(s) il va répondre.

2. Environnement de Test

- **Version de l'API (Backend Go) :** vX.X.X
- **Version de l'App (React Native) :** vX.X.X
- **Équipements IoT connectés :**
 - 1x Clé Sonoff Zigbee 3.0
 - Lister les équipements de test : ex. 1x Ampoule Zigbee, 1x Caméra IP RTSP
- **Réseau :** Testé en local (WiFi) et en distant (4G via Traefik / Port 443).

3. Cahier des recettes

Vue d'ensemble de la couverture de test.

ID Exigence	Description Spécification	ID Test	Status	Raison de l'echec
SF-1.1	Connexion sécurisée de l'utilisateur	UAT-001	a tester/	
SF-5.1	Gestion des volets (pourcentage)	UAT-002	validé	
ST-2.1.3	Authentification JWT / RBAC	UAT-003	echoué	

4. Fiches de Tests d'Acceptation (UAT user acceptance test)

Test UAT-001 : Authentification de l'utilisateur

- **Exigence liée :** SF-1.1 / ST-2.1.3
- **Type de test :** End-to-End (Application Mobile)
- **Pré-requis :** L'application est installée sur le smartphone, la base de données PostgreSQL contient un utilisateur actif avec les identifiants `admin@domotique.local` / `Password123!`.

Scénario Gherkin (BDD) :

Étant donné que je suis sur l'écran d'authentification de l'application mobile
Quand je saisis l'email "admin@domotique.local" et le mot de passe "Password123!"
Et que j'appuie sur le bouton "Se connecter"
Alors je suis redirigé vers mon plateau principal
Et un token JWT est stocké de manière sécurisée sur le téléphone.

Étapes d'exécution :

1. Lancer l'application React Native.
2. Entrer l'email et le mot de passe.
3. Cliquer sur "Se connecter".

Résultat de l'exécution :

- ☐ **Pass** (Succès)
- ☐ **Fail** (Échec)
- ☐ **Blocked** (Bloqué)
 - **Date d'exécution** : [Date]
 - **Commentaires / Bugs relevés** : *[Ajouter des notes si échec, ex: "Le bouton de connexion ne réagit pas sur iOS."]*

Test UAT-002 : Contrôle d'ouverture d'un volet électrique

- **Exigence liée** : SF-5.1
- **Type de test** : End-to-End (Matériel & Logiciel)
- **Pré-requis** : L'utilisateur est connecté, un volet connecté est appairé sur Zigbee2MQTT et assigné au panel de l'utilisateur.

Scénario Gherkin (BDD) :

Étant donné que je suis sur le panel de gestion du "Salon"
Quand je règle le curseur (slider) du volet sur "50%"
Alors l'application affiche l'état "En mouvement"
Et le volet physique s'arrête exactement à la moitié de sa course.

Étapes d'exécution :

1. Naviguer sur le panel "Salon".
2. Glisser le slider du volet de 0% à 50%.
3. Observer l'interface et le comportement du relais physique.

Résultat de l'exécution :

- ☐ **Pass**
- ☐ **Fail**
- ☐ **Blocked**
 - **Date d'exécution** : [Date]
 - **Commentaires / Bugs relevés** : [...]

5. Bilan et Signatures

Résumé de la campagne de tests :

- Total des tests : [X]
- Tests réussis : [X]
- Tests échoués : [X]
- Taux de succès : [X]%

Décision de Release (Go / No-Go) : [] GO / [] NO-GO

Exemple de test unitaire attendu sur l'API GO, consommatrice du serveur MQTT et de ses topics

```
func TestingShutterSignal(t *testing.T) {  
    mockClient := &MockMqttClient{}  
    topicAttendu := "zigbee2mqtt/shutter_living-room/set"  
    payloadAttendu := `{"position":50}`  
  
    commande := map[string]int{"position": 50}  
    mockClient.Publish("zigbee2mqtt/shutter_living-room/set", commande)  
  
    assert.NoError(t, err, "L'envoi ne devrait pas générer d'erreur")  
  
    mockClient.AssertExpectations(t)  
}
```

Stratégie de maintenance :

Les utilisateurs possèdent une installation locale du server raspberry et n'envoie aucune données vers nos serveur. La seule donnée que nous conservons est l'acte d'achat avec les configurations utilisé par l'utilisateur.

En cas de besoin l'utilisateur peut contacter le support de l'application. Le support pourra facilement identifier l'utilisateur à l'aide de son adresse et de son contact en base de données.

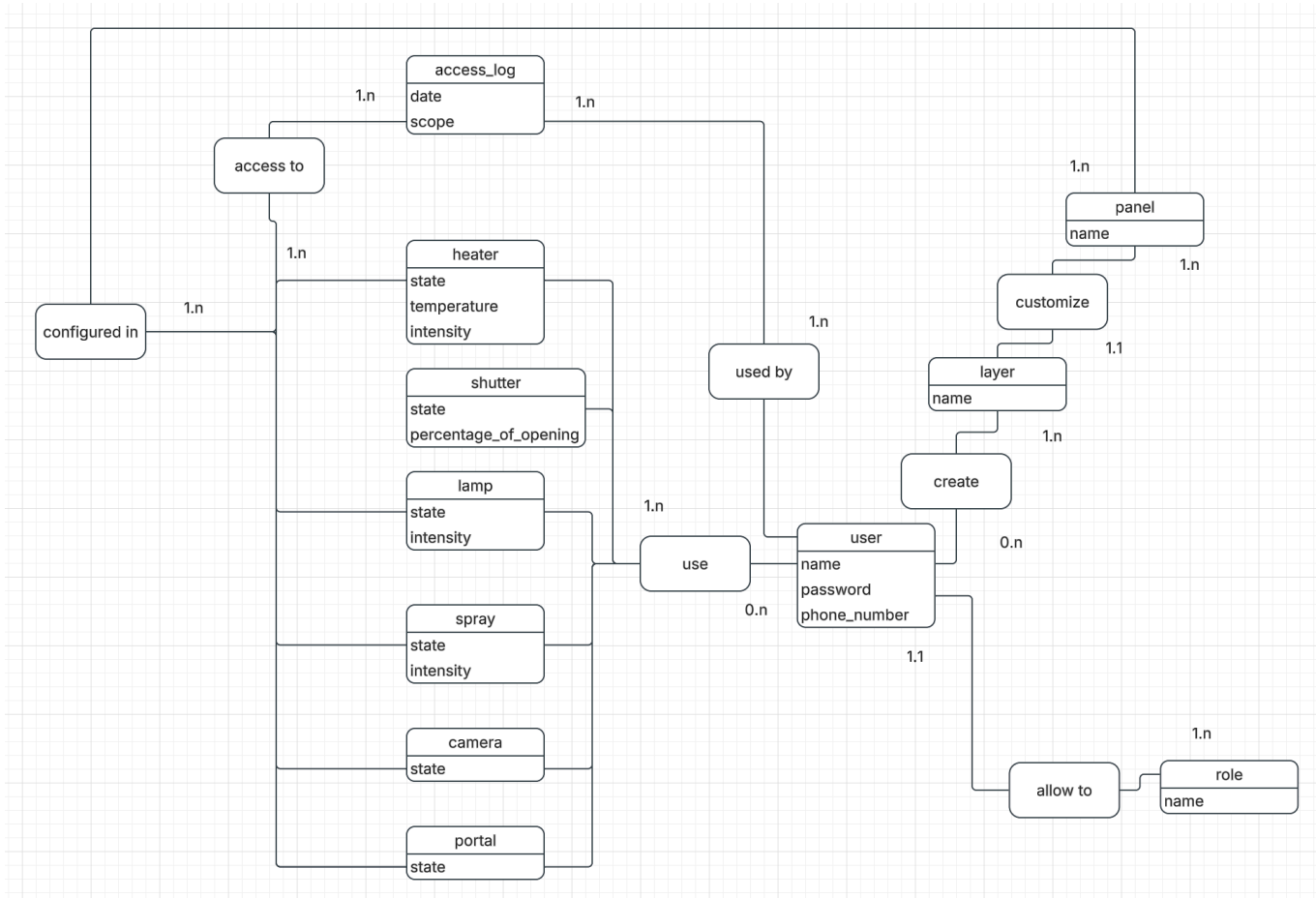
Le support et le client devrons se mettre d'accord sur un rendez-vous technique durant lesquels nos développeur pourront débogger son application.

Le support devra d'abord proposer des mise à jour simple de réinitialisation des données locales ou de certains composants avant de faire intervenir un développeur

En ce qui conserve l'intervention de technicien en domotique, nous ne sommes pas concerné

Base de données

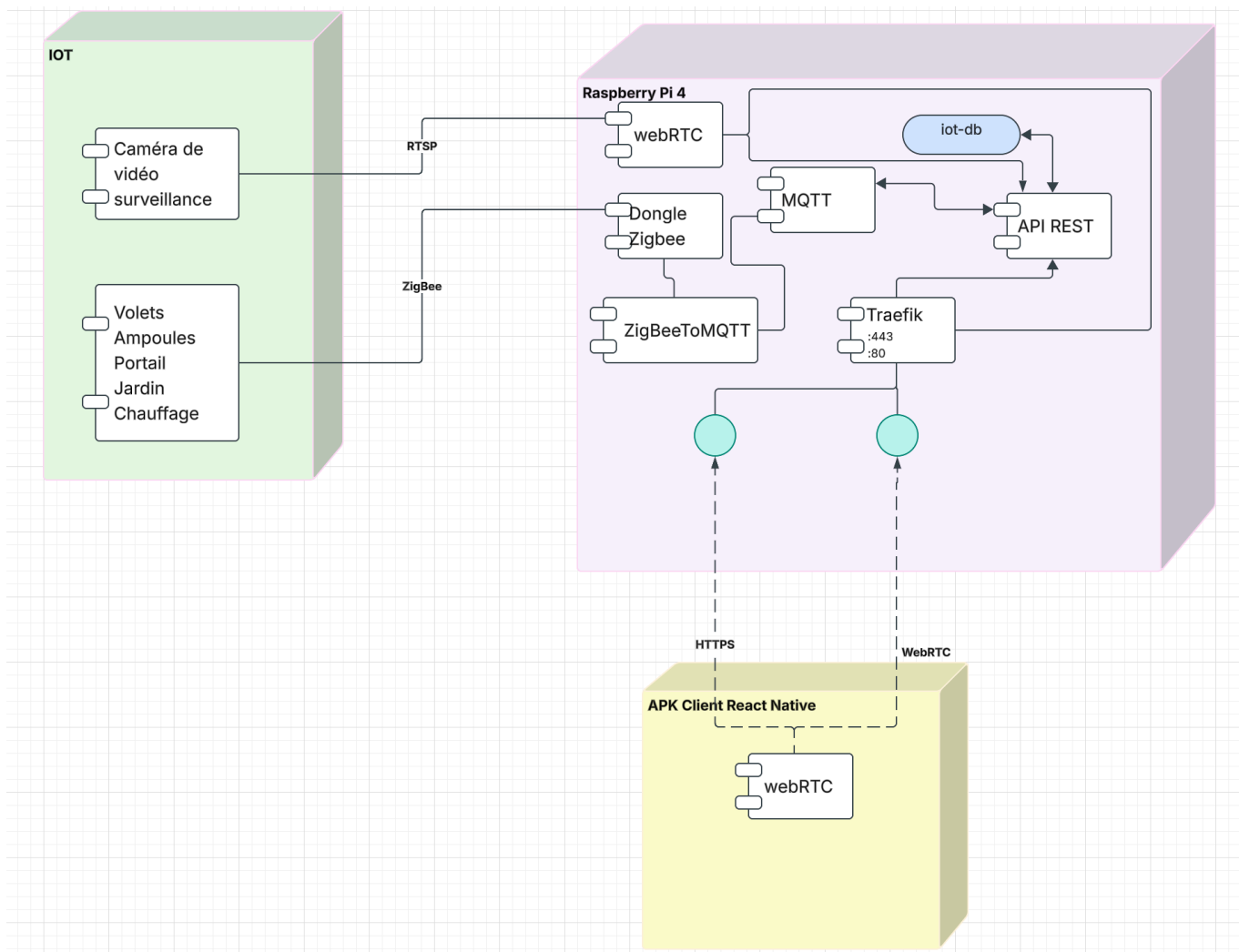
MCD



Conception détaillée

UML

Diagramme de déploiement



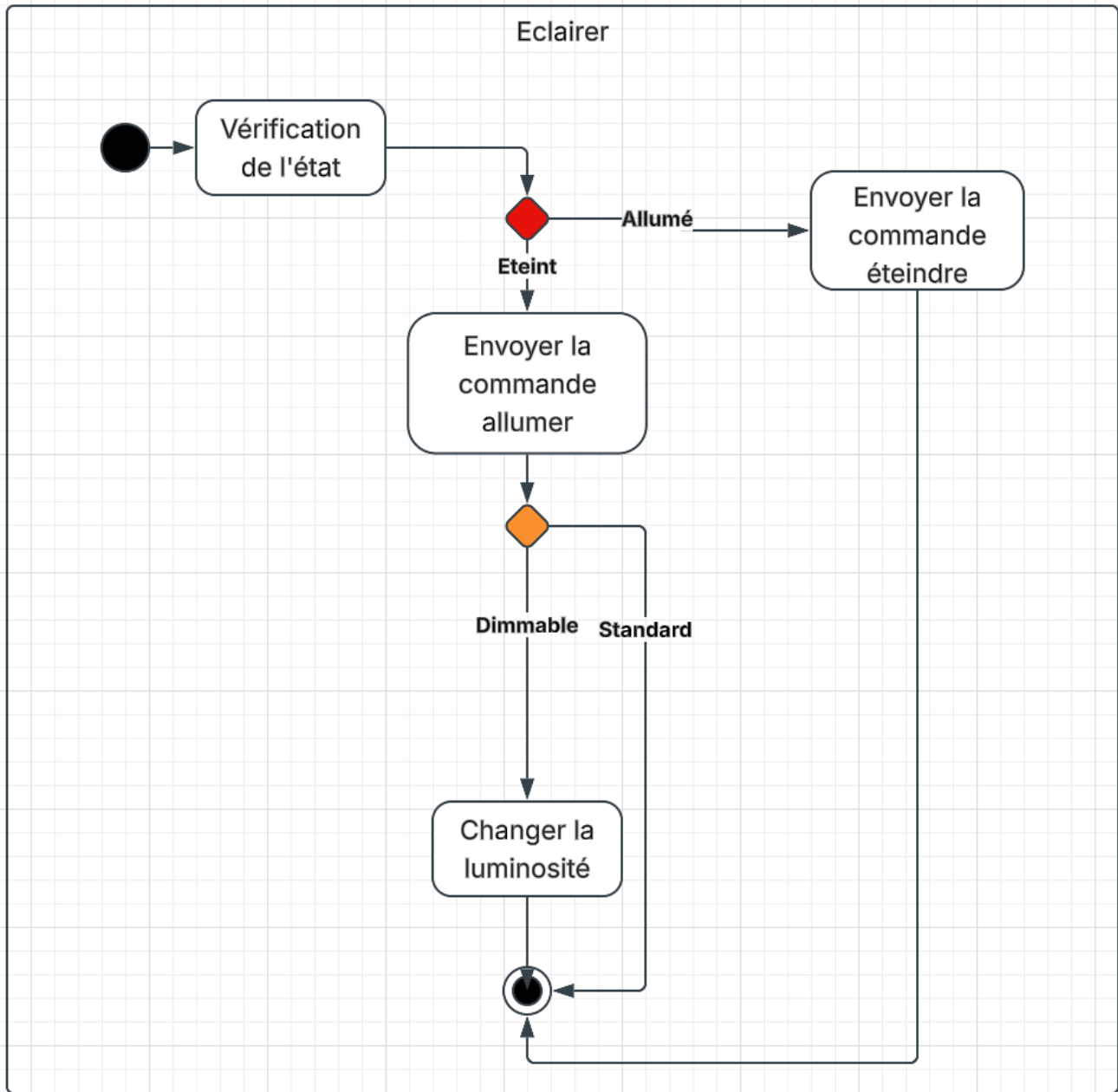
IOT est relié au Raspberry PI4 qui est l'hébergeur central de l'application
 Les caméras de surveillance communiquent en RTSP pour pouvoir atteindre un flux transmissible en webRTC pour l'application mobile.

L'IOT peut atteindre la stack à travers le Dongle ZigBee qui est l'hardware chargé de recueillir les connexion utilisant le protocole de communication ZigBee. Les communication intercepté seront directement renvoyé au serveur ZigBeeToMqtt avec de pouvoir exploiter les informations en JSON. Celles-ci seront stockées dans le serveur (Broker) MQTT. L'API REST pourra a son tour consommer les informations contenues dans les topics du serveur MQTT et émettre des instruction pour reconfigurer les appareil IOT en parcourant le sens inverse à travers tous les composants évoqués précédemment.

Enfin, L'API REST stockera le monitoring du flux WebRTC pour historiser les accès ainsi que les commandes et configuration récurrentes effectuées sur l'API REST

Le parcours de ces données sont sécurées a travers le serveur HTTPS Traefik et exposées pour l'application REACT Native

Diagrammes d'état transition



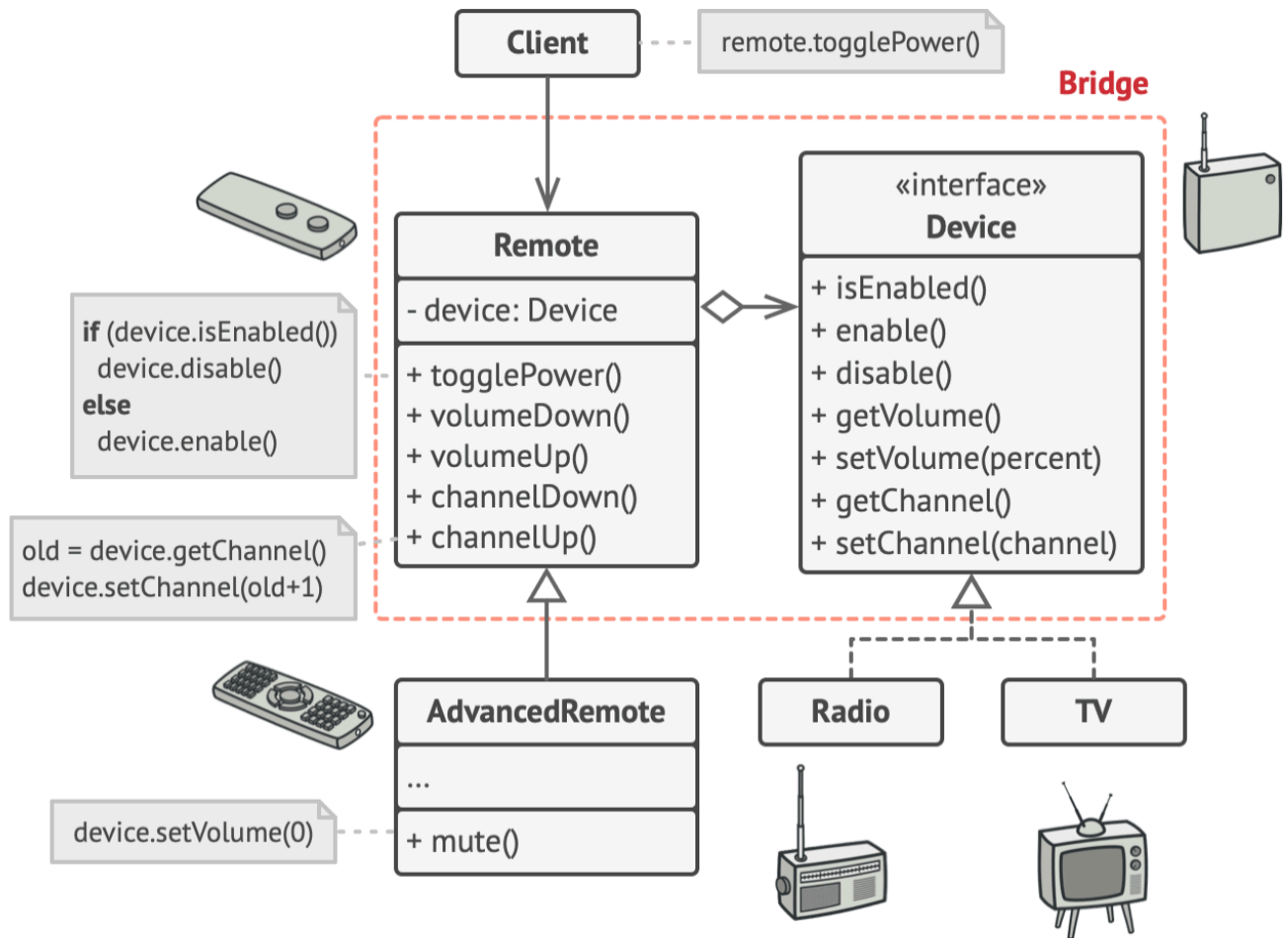
Voici le cycle de vie qui a lieu durant l'éclairage/Eteindre une ou plusieurs ampoules.

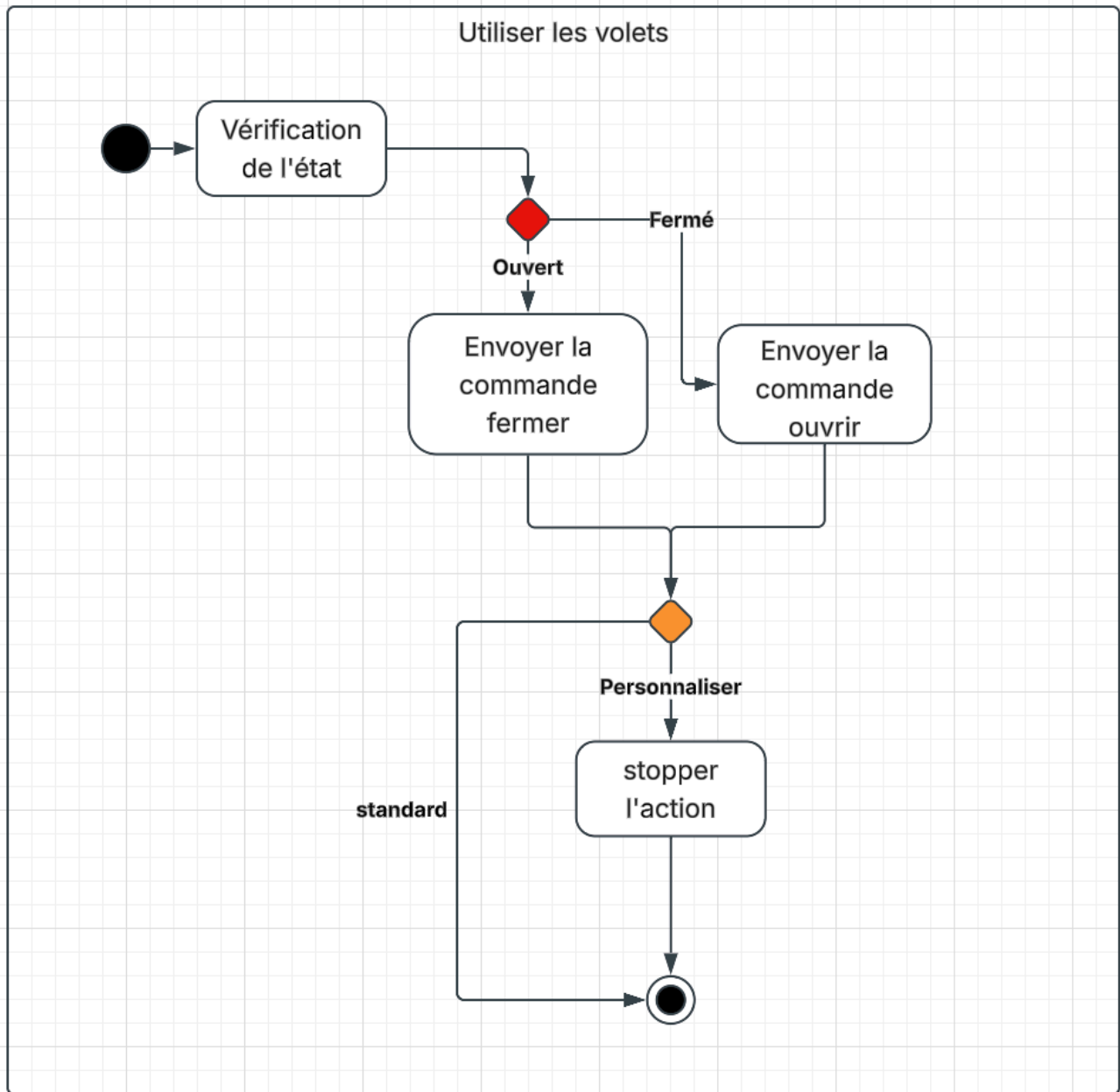
Le système vérifie l'état d'un éclairage pour proposer la commande inverse et ainsi permuter son état. S'il s'agit d'un éclairage et que l'ampoule réagit aux variations de luminosité, elle passera par l'état optionnel, changer la luminosité.

Ce procédé fonctionne par la gestions de signaux industrialisés du matériel connecté. Il s'agira seulement d'utiliser le bon signal plutôt que de le programmer entièrement.

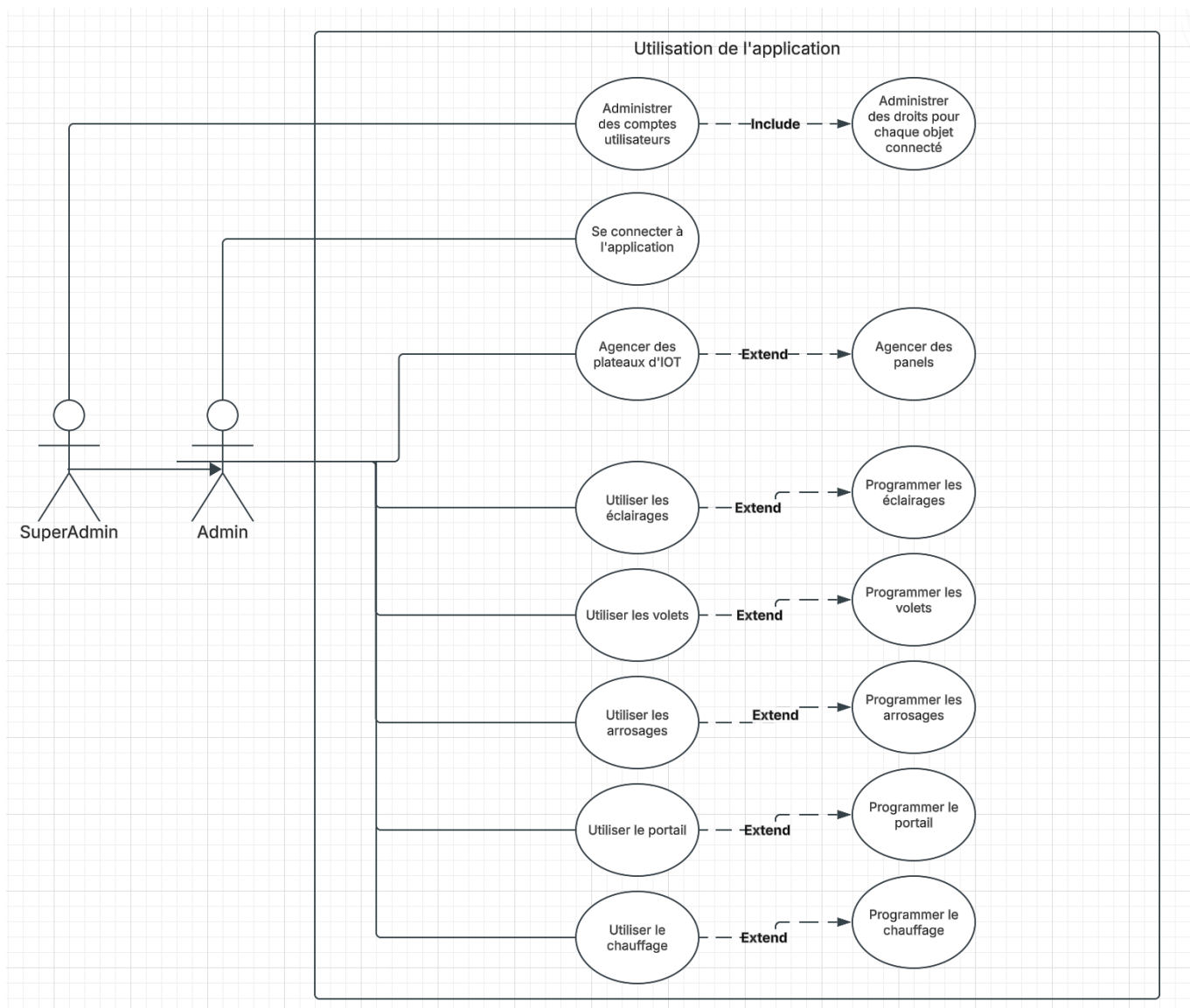
Voici un exemple connu du site refactoring.guru permettant de mieux comprendre l'implémentation industrielles des matériels entre eux, sans avoir besoin de connaître leur implémentation pour être utilisé à travers divers scénarios

Design pattern Bridge





Les volets devront pouvoir être fermé ou bien ouvert s'il on appuie sur le bouton stop ou bien s'il l'ont décidé d'un pourcentage de fermeture ou d'ouverture



L'administration des droits est obligatoire pour chaque nouveau compte

En fonction de ses droits, l'admin aura accès a certains objets connectés et a leur programmation dans le temps

Petite erreur sur les extends, les flèches devraient pointer dans l'autre sens, vers le premier cas d'utilisation

Planning Agile avec sprint de 2 semaines

	État	Person...	September	October
☐ > Tickets Sprints			SCRUM Sprint 1 SCRUM Sprint 2 SCRUM Sprint 3 SCRUM Sprint 4 SCRUM Sprint 5	
☐ > SCRUM-6 Infrastructure & Sécurité	██████████	N...	██████████	
☐ > SCRUM-11 Gestion des Utilisateurs et Droits	██████████	N...		██████████
☐ > SCRUM-15 Pilotage Domotique	██████████	N...		██████████
☐ > SCRUM-20 Vidéosurveillance et Monitoring	██████████	N...		██████████
+ Créer		4 sur 4		

☐ **SCRUM Sprint 5** 25 oct. – 8 nov. (3 tickets)

1600Démarrer un sprint...

Mise en place du monitoring et des flux webRTC

☒ SCRUM-23

En tant qu'Admin, je veux consulter les logs d'accès et d'utilisation du ...

Vidéosurveillance ...

À faire ▾

4

=

☒ SCRUM-22

En tant que système, je veux intercepter les appels vidéo WebRTC via ...

Vidéosurveillance ...

À faire ▾

6

=

☒ SCRUM-21

En tant qu'utilisateur, je veux visionner le flux de ma caméra en direct s...

Vidéosurveillance ...

À faire ▾

6

=

+ Créer



3 tickets visibles sur 3 | Estimation : 16 sur 16

Les daily seront l'occasion d'échanger sur les tickets nécessitant le plus de pair programming afin de ne pas bloquer sur l'implémentation des différentes technologies